

Python Basics - Syntax Notes

1. Printing Output

```
print("Hello, World!")
```

2. Comments

- **Single-line comment**

```
# This is a comment
```

- **Multi-line comment**

```
"""
This is a multi-line comment.
Used for documentation.
"""
```

3. Variables and Data Types

```
x = 10          # Integer
y = 3.14        # Float
name = "John"   # String
is_python = True # Boolean
```

4. Taking User Input

```
name = input("Enter your name: ")
print("Hello, " + name)
```

5. Conditional Statements

```
age = 18

if age >= 18:
    print("You are an adult.")
elif age > 12:
    print("You are a teenager.")
else:
    print("You are a child.")
```

6. Loops

- **For Loop**

```
for i in range(5):  
    print(i)    # Prints 0 to 4
```

- **While Loop**

```
x = 0  
while x < 5:  
    print(x)  
    x += 1
```

7. Lists (Arrays)

```
fruits = ["apple", "banana", "cherry"]  
print(fruits[0]) # Output: apple  
fruits.append("mango") # Add new item
```

8. Functions

```
p  
def greet(name):  
    return "Hello, " + name  
  
print(greet("Alice"))
```

9. Dictionaries (Key-Value Pairs)

```
person = {"name": "John", "age": 25}  
print(person["name"]) # Output: John
```

10. Exception Handling

```
try:  
    x = 10 / 0  
except ZeroDivisionError:  
    print("Cannot divide by zero!")
```

Python File Handling

Python allows you to **read, write, and manipulate files** using built-in functions.

1. Opening a File

Use the `open()` function:

```
file = open("example.txt", "r") # Open file in read mode
```

Modes:

- `"r"` → Read (default)
 - `"w"` → Write (overwrites file)
 - `"a"` → Append (adds to the end)
 - `"x"` → Create (if file does not exist)
-

2. Reading a File

```
file = open("example.txt", "r")
content = file.read()
print(content)
file.close() # Always close the file
```

- **Read a single line:** `file.readline()`
 - **Read all lines as a list:** `file.readlines()`
-

3. Writing to a File

```
file = open("example.txt", "w")
file.write("Hello, World!\n") # Overwrites existing content
file.close()
```

To **append** instead of overwriting:

```
file = open("example.txt", "a")
file.write("This is new content.\n")
file.close()
```

4. Using `with` (Best Practice)

```
with open("example.txt", "r") as file:
    content = file.read()
    print(content)  # File closes automatically
```

5. Checking if a File Exists

```
import os

if os.path.exists("example.txt"):
    print("File exists!")
else:
    print("File not found!")
```

6. Deleting a File

```
import os

if os.path.exists("example.txt"):
    os.remove("example.txt")
    print("File deleted.")
else:
    print("File does not exist.")
```

Python Tuples

A **tuple** is an **ordered, immutable** collection.

1. Creating a Tuple

```
# Tuple with multiple elements
my_tuple = (1, 2, 3, 4)

# Tuple with one element (must include a comma)
single_tuple = (5,)

# Tuple with different data types
mixed_tuple = (10, "Hello", 3.14)

print(my_tuple)
print(single_tuple)
print(mixed_tuple)
```

2. Accessing Tuple Elements

```
print(my_tuple[0])  # First element
print(my_tuple[-1]) # Last element
```

3. Tuple Operations

```
tuple1 = (1, 2, 3)
tuple2 = (4, 5)

# Concatenation
new_tuple = tuple1 + tuple2 # (1, 2, 3, 4, 5)

# Repetition
repeat_tuple = tuple1 * 2 # (1, 2, 3, 1, 2, 3)

print(new_tuple)
print(repeat_tuple)
```

4. Unpacking Tuples

```
my_tuple = (1, 2, 3)
a, b, c = my_tuple
print(a, b, c) # Output: 1 2 3
```

5. Tuple Methods

```
my_tuple = (1, 2, 3, 2, 2, 4)

print(my_tuple.count(2)) # Count occurrences of 2
print(my_tuple.index(3)) # Find index of 3
```

Python Sets

A **set** is an **unordered, mutable** collection with **no duplicate values**.

1. Creating a Set

```
my_set = {1, 2, 3, 4} # Set with values
empty_set = set() # Creating an empty set
print(my_set)
```

2. Adding & Removing Elements

```
my_set.add(5) # Add an element
my_set.remove(2) # Remove an element (Error if not found)
my_set.discard(10) # Remove element (No error if not found)
my_set.pop() # Remove a random element
my_set.clear() # Empty the set
```

```
print(my_set)
```

3. Set Operations

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

# Union (combine all elements)
print(A | B)  # {1, 2, 3, 4, 5, 6}

# Intersection (common elements)
print(A & B)  # {3, 4}

# Difference (elements in A but not in B)
print(A - B)  # {1, 2}

# Symmetric Difference (elements in A or B but not both)
print(A ^ B)  # {1, 2, 5, 6}
```

4. Checking Membership

```
my_set = {1, 2, 3, 4}
print(3 in my_set)  # True
print(5 in my_set)  # False
```

5. Set Methods

```
A = {1, 2, 3}
B = {2, 3, 4}

print(A.issubset(B))  # False
print(A.issuperset(B))  # False
print(A.isdisjoint(B))  # False (They have common elements)
```

Python Lists

A **list** is an **ordered, mutable** collection that allows duplicate values.

Creating a List

```
# List with multiple elements
my_list = [1, 2, 3, 4, 5]
```

```
# List with different data types
mixed_list = [10, "Hello", 3.14, True]

# Empty list
empty_list = []

print(my_list)
print(mixed_list)
print(empty_list)
```

Accessing List Elements

```
my_list = [10, 20, 30, 40, 50]
print(my_list[0])    # First element (10)
print(my_list[-1])   # Last element (50)
print(my_list[1:4])  # Slicing (20, 30, 40)
```

Modifying a List

```
my_list = [1, 2, 3, 4, 5]
my_list[2] = 100    # Change the third element
print(my_list)      # [1, 2, 100, 4, 5]
```

Adding Elements

```
my_list = [1, 2, 3]
my_list.append(4)    # Add at the end
my_list.insert(1, 100) # Insert at index 1
my_list.extend([5, 6]) # Extend list
print(my_list)      # [1, 100, 2, 3, 4, 5, 6]
```

Removing Elements

```
my_list = [10, 20, 30, 40, 50]
my_list.remove(30)   # Remove specific value
my_list.pop(1)       # Remove by index (20)
last_item = my_list.pop() # Remove last item (50)
my_list.clear()      # Remove all elements
print(my_list)      # []
```

Looping Through a List

```
my_list = [1, 2, 3, 4, 5]
```

```
# Using a for loop
for item in my_list:
    print(item)

# Using list comprehension
squared = [x**2 for x in my_list]
print(squared)  # [1, 4, 9, 16, 25]
```

List Operations

```
a = [1, 2, 3]
b = [4, 5, 6]
# Concatenation
print(a + b)  # [1, 2, 3, 4, 5, 6]
# Repetition
print(a * 2)  # [1, 2, 3, 1, 2, 3]
```

List Methods

```
my_list = [3, 1, 4, 1, 5, 9]
print(len(my_list))  # Length of list
print(my_list.count(1))  # Count occurrences of 1
print(my_list.index(4))  # Find index of 4
my_list.sort()  # Sort list
my_list.reverse()  # Reverse list
print(my_list)
```

Copying a List

```
original = [1, 2, 3]
copy1 = original.copy()
copy2 = original[:]  # Slicing method
print(copy1)
print(copy2)
```

Checking Membership

```
my_list = [10, 20, 30, 40]
print(20 in my_list)  # True
print(50 in my_list)  # False
```

Python Dictionaries

A **dictionary** in Python is an **unordered, mutable** collection that stores data in **key-value pairs**.

Creating a Dictionary

```
# Empty dictionary
my_dict = {}
# Dictionary with values
student = {
    "name": "Alice",
    "age": 25,
    "grade": "A"
}
print(student)
```

Accessing Dictionary Values

```
student = {"name": "Alice", "age": 25, "grade": "A"}
print(student["name"])    # Alice
print(student.get("age")) # 25
# Using .get() avoids errors if key doesn't exist
print(student.get("city", "Not Found")) # Not Found
```

Adding & Updating Values

```
student["city"] = "New York" # Add new key-value pair
student["age"] = 26          # Update existing key
print(student)
```

Removing Elements

```
student = {"name": "Alice", "age": 25, "grade": "A"}
student.pop("age")      # Removes a key
del student["grade"]    # Another way to remove a key
student.clear()         # Clears all items
print(student)          # {}
```

Looping Through a Dictionary

```
student = {"name": "Alice", "age": 25, "grade": "A"}
```

```
# Loop through keys
for key in student:
    print(key, ":", student[key])
# Loop through values
for value in student.values():
    print(value)
# Loop through key-value pairs
for key, value in student.items():
    print(f"{key}: {value}")
```

Dictionary Methods

```
student = {"name": "Alice", "age": 25, "grade": "A"}
print(len(student))          # Get number of key-value pairs
print(student.keys())        # Get all keys
print(student.values())      # Get all values
print(student.items())       # Get all key-value pairs
student_copy = student.copy() # Copy dictionary
```